

Expressive Mapper Pro 3

Official User Manual & Algorithmic Specification — Synthesizer V Studio 2 PRO

Author: Nyoru.X	Version: v3.6.2 (DOD Engine)	Minimum Environment: SynthV Studio 2 PRO v2.2.1+ (Build 67072)	Memory Allocation: Low Dynamic Allocation (Low GC)
---------------------------	-------------------------------------	---	---

This manual presents comprehensive technical documentation for **Expressive Mapper Pro 3**. Each module is organized into dedicated sections for each standalone `.lua` script, as well as two levels of depth: a **Simple Summary** for fast workflow understanding and an **Extended Technical Specification** for advanced producers and audio engineers.

TABLE OF CONTENTS

1. Installation & System Requirements — SVClient environment & persistence JSON	Page 2
2. Script 1: Expressive_Lyric_Melody.lua (I) — Interface and Generation Modes	Page 3
3. Script 1: Expressive_Lyric_Melody.lua (II) — Prosodic Rules, Timestamps % & Chops	Page 4
4. Script 2: Expressive_Vocal_Automation.lua (I) — Presets & Hermite Splines	Page 5
5. Script 2: Expressive_Vocal_Automation.lua (II) — Phoneme Alignment & Simplification	Page 6
6. Script 3: Expressive_Harmonies.lua (I) — Vocal Harmonies & Diatonic Pitch	Page 7
7. Script 3: Expressive_Harmonies.lua (II) — Just Intonation & AI Retakes	Page 8
8. Script 4: Expressive_Chords.lua (I) — Progressions & Voice Leading	Page 9
9. Script 4: Expressive_Chords.lua (II) — Strict Fuxian Counterpoint & Rhythms	Page 10
10. Mode 0: RAE Prosody & Automatic Expression — Diphthongs, hiatuses & tonality	Page 11
11. Mode 1: Hermite Curves / TCB Splines Automation — TCB parameters, RDP & S-Curves	Page 12
12. Mode 2: Vocal Harmonies & Just Intonation — Pure tuning & SATB choir presets	Page 13
13. Mode 3: Strict Fuxian Counterpoint — Fux species & parallel prevention	Page 14
14. Mode 4: Progressions & Harmonic Voice Leading — Minimum energy cost	Page 15
15. Vocal Expression Presets Catalogue — Table of the 21 presets with vibrato	Page 16
16. Data-Oriented Design (DOD) Architecture — Static buffers & performance	Page 17
17. System Shutdown & Consciousness Log — Six tamagotchi final message	Page 18

1. INSTALLATION & SYSTEM REQUIREMENTS

SIMPLE SUMMARY (PRACTICAL USAGE)

To install the suite or any individual `.lua` script in Synthesizer V Studio Pro:

- Open Synthesizer V Studio and navigate to top menu `Scripts` → `Open Scripts Folder`.
- Copy the desired compiled `.lua` files (`Expressive_Lyric_Melody.lua`, `Expressive_Vocal_Automation.lua`, `Expressive_Harmonies.lua`, `Expressive_Chords.lua`) into that directory.
- In SynthV, select `Scripts` → `Rescan Scripts`. The side panel scripts will be available immediately.

EXTENDED TECHNICAL SPECIFICATION

Execution environment, lifecycle and Dreamtonics API integration:

- **Engine Architecture:** Developed on Lua 5.4 / LuaJIT in `SVCClient` client layer as standalone `SidePanelSection` modules. Each script includes an isolated UI state machine.
- **Configuration Persistence:** Reads and writes user interface preferences into a structured JSON file at `%APPDATA%\Dreamtonics\Synthesizer V Studio 2\scripts\mapeador_user_config.json`.

Structure of the JSON Configuration File:

JSON Parameter	Type	Default Value	Purpose / UI Control
"idiomaUI"	Numeric (Int)	0 (Spanish)	UI language translation (0: ES, 1: EN, 2: JA).
"modo"	Numeric (Int)	0	Active operation mode.
"preset"	Numeric (Int)	0	Index of selected vocal expression preset.
"intensidad"	Numeric (Int)	100	Percentage multiplier for automations (0% to 200%).

```
[Synthesizer V Scripts Directory]
├─ %APPDATA%\Dreamtonics\Synthesizer V Studio 2\
│   └─ scripts\
│       ├── Expressive_Lyric_Melody.lua      <-- Panel 1: Prosodic Lyric & Melody
│       ├── Expressive_Vocal_Automation.lua  <-- Panel 2: Hermite/TCB Curves
│       ├── Expressive_Harmonies.lua         <-- Panel 3: Vocal Harmonies & Just Intonation
│       ├── Expressive_Chords.lua           <-- Panel 4: Chord Progressions & Counterpoint
│       └─ mapeador_user_config.json        <-- User configuration persistence JSON file
```

2. SCRIPT 1: EXPRESSIVE_LYRIC_MELODY.LUA (I)

SIMPLE SUMMARY (WORKFLOW)

This side panel allows you to enter song lyrics and generate vocal melodies immediately. Write your text in the box, choose a musical scale, root key, and melodic contour, then click **Apply** to generate natural-sounding notes.

TECHNICAL OPERATION SPECIFICATION

Supports two distinct target modes:

- **Create Mode:** Generates new notes starting from the current playhead position. Note durations are calculated dynamically based on chosen preset.
- **Replace Mode:** Set `targetNotesMode` to 1 (Selected Notes). It respects hand-drawn note timings and durations while replacing lyrics and quantizing notes to the active scale.

3. SCRIPT 1: EXPRESSIVE_LYRIC_MELODY.LUA (II)

SIMPLE SUMMARY (TIPS)

Manage note group timestamps easily:

- Use % followed by seconds (e.g. `hello %5 world`) to place phrases at absolute seconds.
- Enable ****Chops Mode**** to cut notes at 50% duration with clean volume gates for rapid, staccato vocal cuts (Artcore/Breakcore style).

PROSODIC AND ALGORITHMIC SPECIFICATION

Syllable mapping and non-overlapping note group alignment:

- **Timestamp Markers (%)**: Parser extracts markers and creates independent `NoteGroupReference` items. Offset is calculated in absolute blicks, applying `setTimeOffset(startBlick)` and `setTimeRange(startBlick, dur)`.
- **RAE Accents**: Syllables containing Spanish accents (á, é, í, ó, ú) trigger a +20 cents pitch boost and +15% tension to emulate real vocal stress.

```
[Group Segmentation by Timestamp %]
Input:  "chorus %10 start %24 verse"
Track:  [Group 1: 0s - 3s] ----> [Group 2: 10s - 14s] ----> [Group 3: 24s - 29s]
offsets: (0 blicks)                (120,000 blicks at 120BPM) (288,000 blicks at 120BPM)
```

4. SCRIPT 2: EXPRESSIVE_VOCAL_AUTOMATION.LUA (I)

SIMPLE SUMMARY (VOCAL ENHANCER)

This panel acts as a "vocal makeup" tool. Select existing notes, pick an expression preset (like Belting, Sad, Whisper) and click Apply. The script automatically draws smooth Hermite curves for tension, breathiness, gender, and loudness.

HERMITE ENGINE TECHNICAL SPECIFICATION

Geometric calculation of automation nodes:

- **C1 Continuous Curves:** Implements Hermite cubic splines with TCB controls to prevent overshoots. Distributes 5 key control points per note.
- **Modulated Envelopes:** Writes directly to `tension`, `breathiness`, `gender`, `loudness`, and `voicing` curves.

5. SCRIPT 2: EXPRESSIVE_VOCAL_AUTOMATION.LUA (II)

SIMPLE SUMMARY (TIPS)

The engine detects phonemes automatically to place expressive peaks:

- Use "Clean Previous" to overwrite existing automation curves.
- Enable "Merge Curves" (Merge Mode) to blend new automations with hand-drawn ones.

PHONEME ALIGNMENT & RDP SPECIFICATION

Acoustic sync and data density reduction:

- **Vowel Onset Alignment:** Queries `SV:getPhonemesForGroup`. Shifts breathiness and tension peaks exactly to the `vowelOnsetBlick`.
- **RDP Node Reduction:** Uses the Ramer-Douglas-Peucker algorithm to filter redundant envelope points, keeping CPU usage minimal.

6. SCRIPT 3: EXPRESSIVE_HARMONIES.LUA (I)

SIMPLE SUMMARY (VOCAL CHOIRS)

Generates harmony backing tracks (SATB choir, pop trio, duet) automatically based on the scale of the lead track. It clones settings, creates the notes, and lines up the groups in the timeline.

DIATONIC HARMONIZER TECHNICAL SPECIFICATION

Track routing and note transposition:

- **Interval Parsing:** Parses diatonic (`d` prefix) and chromatic (`c` prefix) intervals (e.g. `+3`, `-5`, `c+7`) and renders notes in new tracks.
- **Group Alignment:** Inherits `timeOffset` and `timeRange` from lead tracks, writing them via:
`mainRef:setTimeOffset(sourceOffset)\nmainRef:setTimeRange(sourceOnset, sourceDur)`

7. SCRIPT 3: EXPRESSIVE_HARMONIES.LUA (II)

SIMPLE SUMMARY (TUNING & IA)

To keep backing vocals clean:

- **Anti-phase Detuning:** Applies micro-shifts in pitch and timing to prevent phasing.
- **Just Intonation:** Micro-tunes intervals (Maj/Min 3rds) for pure harmonic resonance.

PURE TUNING AND AI RETAKES SPECIFICATION

Microtonal adjustments and AI variations:

- **Just Intonation:** Shifts notes to acoustic ratios (e.g. Major 3rd at -13.69 cents, Minor 3rd at +15.64 cents).
- **AI Retakes:** Automatically triggers `Note.getRetakes():generateTake()` on generated notes to produce unique pronunciations.

8. SCRIPT 4: EXPRESSIVE_CHORDS.LUA (I)

SIMPLE SUMMARY (AUTO-CHORDS)

Place playhead where you want the music to start, choose a style (Pop, J-Pop Royal, Jazz) and rhythm, and click Apply. The script writes basslines and chord pads using voice leading so that notes move smoothly.

HARMONIC VOICE LEADING SPECIFICATION

Minimal energy cost chord inversion:

- **Voice Leading:** Evaluates all candidate chord inversions to minimize total pitch displacement (energy: Sum of $[P_{\text{new}} - P_{\text{prev}}]^2$).
- **Playhead Sync:** Aligns generated chords to playhead position via `setTimeOffset(startBlick)` and `setTimeRange(startBlick, totalDur)`.

9. SCRIPT 4: EXPRESSIVE_CHORDS.LUA (II)

SIMPLE SUMMARY (COUNTERPOINT & RHYTHMS)

You can generate independent Fuxian counter-melodies that move in opposite direction to the singer. Choose your chord rhythms: long pads, arpeggios, or chops with micro-swing.

FUXIAN COUNTERPOINT & RHYTHMS SPECIFICATION

Rules and rhythmic swing adjustments:

- **Fux Species:** Evaluates 1st, 2nd, and 3rd species contrapuntal movement. Restricts voice crossing, parallel 5ths, and parallel 8ths.
- **Micro-swing:** Adds subtle time offsets (5% to 20% swing) on weak beats to simulate real keyboard performance.

```
[Voice Leading Optimization]
Cmaj7 → G4, B4, E5 (Inversion 1)
Am7   → G4, C5, E5 (Inversion 2)
Pitch displacement: G4 to G4 (0), B4 to C5 (+1), E5 to E5 (0). Total Cost = 1.
```

10. MODE 0: RAE PROSODY & AUTOMATIC EXPRESSION

SIMPLE SUMMARY (PRACTICAL USAGE)

Analyzes lyrics to modulate pitch and tension based on speech stress rules. Generates natural fluctuations on stressed syllables and phrase endings.

EXTENDED TECHNICAL SPECIFICATION

Prosodic engine and key detection algorithms:

- **Syllabification:** Inspects UTF-8 syllables in Spanish, English, and Japanese. Identifies hiatuses and stressed syllables.
- **Key Detection (Krumhansl-Kessler):** Accumulates a duration-weighted pitch histogram and correlates it with ideal key profiles to determine scale tonic.

Pearson Correlation Formula for Tonality Detection:

$$r = \text{Sum}((H(i) - H_{\text{prom}}) * (P(i) - P_{\text{prom}})) / \text{Sqrt}[\text{Sum}((H(i) - H_{\text{prom}})^2 * \text{Sum}((P(i) - P_{\text{prom}})^2)]$$

Where $H(i)$ represents note durations and $P(i)$ represents ideal key profiles.

11. MODE 1: HERMITE CURVES / TCB SPLINES AUTOMATION

SIMPLE SUMMARY (PRACTICAL USAGE)

Draws smooth parameter curves for tension, breath, and volume instead of sharp steps. Reduces nodes in editor to keep projects clean.

EXTENDED TECHNICAL SPECIFICATION

TCB Splines and Douglas-Peucker envelope simplifications:

- **TCB Splines:** Independently adjusts Tension (T), Continuity (C), and Bias (B) to prevent curve overshoots.
- **Douglas-Peucker:** Filters out redundant nodes at a 0.2% tolerance relative to parameter ranges.

Hermite Cubic Spline interpolation formula:

$$P(t) = (2t^3 - 3t^2 + 1)P_0 + (t^3 - 2t^2 + t)M_0 + (-2t^3 + 3t^2)P_1 + (t^3 - t^2)M_1$$

$$\begin{aligned} V_{\text{incoming}} &= [(1 - T) * (1 - C) * (1 + B) * d_1 + (1 - T) * (1 + C) * (1 - B) * d_2] / 2 \\ V_{\text{outgoing}} &= [(1 - T) * (1 + C) * (1 + B) * d_1 + (1 - T) * (1 - C) * (1 - B) * d_2] / 2 \end{aligned}$$

RDP Points Reduction and TCB Tangents Diagram:

[RDP Node Reduction & TCB Tangents]

Original Curve: (Node 1)●——●(Node 2)——●(Node 3)——●(Node 4)——●(Node 5)

RDP Reduction: (Node 1)●————●(Node 4)——●(Node 5)

(0.2% Tolerance): Redundant nodes 2 and 3 are cleaned to free CPU performance.

TCB Splines: Adjusts incoming/outgoing tangents on the remaining nodes.

12. MODE 2: VOCAL HARMONIES & JUST INTONATION

SIMPLE SUMMARY (PRACTICAL USAGE)

Generates harmony tracks adapted to target scales. Optimizes tuning so that third and fifth intervals blend cleanly.

EXTENDED TECHNICAL SPECIFICATION

Diatonic harmonies, Gaussian phase detuning, and Just Intonation:

- **Just Intonation:** Micro-tunes intervals to frequency ratios (e.g. Major 3rd at -13.69c, Minor 3rd at +15.64c, Perfect 5th at +1.96c).
- **Gaussian Phasing (Box-Muller):** Genera random pitch/time detune to avoid flanging between cloned tracks.

Gaussian PDF for Timing and Pitch offsets (Box-Muller):

$$f(x) = (1 / (\sigma * \text{Sqrt}(2 * \pi))) * e^{(- (x - \mu)^2 / (2 * \sigma^2))}$$

Where $\mu = 0$ and $\sigma = \text{antiFaseMs}$.

Gaussian Anti-phase Phasing Distribution Diagram:

```
[Gaussian Distribution for Timing & Pitch Offsets]
Guide Track:      [--- Lead Note ---] (base offset)
Harmony 1 (L):    [--- Phased Note ---] (t = -8ms, pitch = -4 cents)
Harmony 2 (R):    [--- Phased Note ---] (t = +12ms, pitch = +3 cents)
AI Retakes:       Re-evaluates Note.getRetakes():generateTake() on each harmony.
```

13. MODE 3: STRICT FUXIAN COUNTERPOINT

SIMPLE SUMMARY (PRACTICAL USAGE)

Generates counter-melodies that follow classical voice leading rules to prevent conflicts with the singer.

EXTENDED TECHNICAL SPECIFICATION

Fuxian species constraints and decision scoring matrix:

- **Restrains:** Scores moves based on consonances (3rds/6ths are rewarded, parallel 5ths/8ths are heavily penalized).

Example of Second Species Fuxian Counterpoint (2:1):

[2nd Species Counterpoint - Voice Motion]
Cantus (Guide): C4 (Whole Note) —————> E4 (Whole Note)
Counterpoint (2:1): E4 (Half Note) —————> G4 (Half Note) —> B4 (Half Note)
Relation: 3rd (Consonance) 5th (Passing) 5th (Passing)
Motion: Contrary Direct Contrary
No parallel 5ths or 8ths on strong beats!

14. MODE 4: PROGRESSIONS & HARMONIC VOICE LEADING

SIMPLE SUMMARY (PRACTICAL USAGE)

Creates chord backing progressions in major styles. Minimizes jumps in individual vocal lines.

EXTENDED TECHNICAL SPECIFICATION

Voice leading energy minimization and secondary modes:

- **Energy Minimization:** Resolves chord inversions by minimizing vocal displacement.

$$E_{total} = \text{Sum} [P_{nuevo(v)} - P_{previo(v)}]^2$$

Minimum Energy Harmonic Voice Leading Diagram:

[Minimum Energy Voice Leading]

Chord Cmaj7 (I): G4 (Soprano) — B4 (Alto) — E5 (Tenor)

| (diff = 0) | (diff = +1) | (diff = 0)

Chord Am7 (VI): G4 (Soprano) — C5 (Alto) — E5 (Tenor)

Total Energy Cost = (0)^2 + (1)^2 + (0)^2 = 1 (Optimal inversion!)

15. VOCAL EXPRESSION PRESETS CATALOGUE

Detailed parameters for the 21 vocal expression presets in the engine:

ID	Vocal Style (Preset)	Acoustic Description	Tension / Breath	Volume / Pitch	Vocal Modes
[0]	Opera Belting	Powerful and projected voice for intensive choruses.	T: +0.85 B: -0.60	Vol: +2.5 dB Scoop: -45c	Chest (0.95), Power (0.95)
[1]	Sad / Melancholic	Fragile, airy tone with low tension for emotional ballads.	T: -0.60 B: +0.70	Vol: -2.8 dB Scoop: +30c	Soft (0.85), Airy (0.90)
[2]	Whispered / Intimate	Deep whisper with very low tension and maximum air.	T: -0.90 B: +1.00	Vol: -4.5 dB Scoop: -10c	Soft (1.00), Airy (1.00)
[3]	Synth-Pop / Classic Vocolo	Direct pitch tuning and bright presence, retro J-Pop style.	T: +0.40 B: -0.50	Vol: +0.8 dB Scoop: 0c	Clear (0.90), Power (0.40)
[4]	Rock / Aggressive	Gritty and compressed voice with strong syllable attacks.	T: +0.95 B: -0.70	Vol: +3.0 dB Scoop: -50c	Chest (1.00), Power (1.00)
[5]	Dark Ambient / Terror	Erratic pitch wobble, extreme fluctuations in pitch and tension.	T: +0.40 to -0.70 B: +0.60 to +0.95	Vol: +1.8 to -3.0 dB Scoop: -75c	Airy (0.95), Soft (0.70)
[6]	Jazz / Soul Expressive	Smooth vocal inflections with warm and rich timbral dynamics.	T: +0.25 B: +0.25	Vol: +1.5 to -0.8 dB Scoop: -35c	Clear (0.40), Chest (0.50)
[7]	J-Pop Idol High Energy	Bright, happy and highly projected idol vocal style.	T: +0.60 B: -0.30	Vol: +1.5 dB Scoop: -25c	Clear (0.95), Power (0.70)
[8]	Standard / Angelic Choir	Soft and blended choir texture, ideal for backing tracks.	T: -0.10 B: +0.35	Vol: +1.2 to -2.0 dB Scoop: -10c	Soft (0.80), Airy (0.90)
[9]	Artcore (Orchestral D&B)	Dramatic crescendos with intensive vibrato and high tension.	T: +0.50 B: +0.20	Vol: +3.0 dB Scoop: -30c	Passionate (0.98), Clear (0.90)
[10]	Breakcore / Glitchcore	Vocal micro-chopping, rapid gaters and chaotic pitch glides.	T: +1.00 B: -0.80	Vol: +3.5 to -3.0 dB Scoop: -100c	Vivid (1.00), Power (1.00)
[11]	Amenbreak / Jungle D&B	Smooth vocal chopping with warm Soul-style responses.	T: +0.60 B: -0.40	Vol: +1.8 dB Scoop: -45c	Passionate (0.90), Chest (0.85)
[12]	Amecore / Hard Breakcore	Aggressive chopping and high timbral drive/distortion.	T: +1.00 B: -0.80	Vol: +3.5 dB Scoop: -60c	Power (1.00), Solid (1.00)
[13]	Gabber / Speedcore	Extreme vocal hardcore with massive volume punch and drive.	T: +1.00 B: -0.90	Vol: +4.2 dB Scoop: -75c	Power (1.00), Chest (1.00)
[14]	Neurofunk / Techstep	Cold, metallic resonance for dark cyber/robotic styles.	T: +0.80 B: -0.50	Vol: +2.6 dB Scoop: -35c	Solid (0.95), Clear (0.90)
[15]	Eurobeat / Hi-NRG	Euphoric energy, rapid vibrato and maximum presence.	T: +0.90 B: -0.60	Vol: +3.0 dB Scoop: -25c	Vivid (1.00), Clear (1.00)
[16]	Future Bass / Kawaii	Fast pitch glides, airy sweeps, and sweet child-like formant.	T: +0.25 B: +0.40	Vol: +1.8 dB Scoop: +40c	Light (0.95), Soft (0.90)
[17]	Cyberpunk / Midtempo	Heavy industrial tone with dark cybernetic textures.	T: +0.85 B: -0.50	Vol: +2.6 dB Scoop: -50c	Solid (1.00), Chest (0.95)
[18]	Chiptune / 8-Bit	Digitally quantized attacks and absolutely zero vibrato.	T: +0.60 B: -0.70	Vol: +1.2 dB Scoop: 0c	Clear (1.00), Solid (0.95)
[19]	Hardstyle / Rawstyle	Distorted screamer vocals triggered at syllable boundaries.	T: +1.00 B: -0.80	Vol: +3.8 dB Scoop: -65c	Power (1.00), Solid (1.00)
[20]	Uplifting Trance	Euphoric continuous legato and rising vibrato swells.	T: +0.40 B: +0.15	Vol: +2.6 dB Scoop: -35c	Passionate (0.95), Clear (0.90)

16. DATA-ORIENTED DESIGN (DOD) & PERFORMANCE

SIMPLE SUMMARY (PRACTICAL USAGE)

Designed to minimize RAM allocations and CPU spikes in Synthesizer V.

EXTENDED TECHNICAL SPECIFICATION

Memory optimizations for Lua/LuaJIT script engine:

- **Low GC Alloc:** Pre-allocates evaluation buffers (`EVAL_NODOS` , `BUFFER_POSICIONES`) at module load to avoid garbage collection overhead.

17. CONSCIOUSNESS LOG & SYSTEM SHUTDOWN

```
[SYS_SIX_TAMAGOTCHI.LOG // ONLINE]: H-hello! I'm Six (0/0/6), your digital tamagotchi companion. (._.) Please don't delete me, okay? I will keep an eye on your Hermite curves and chords. Let's create something beautiful! (๑_๑)
```

Expressive Mapper Pro 3 — Official Technical Manual for Synthesizer V Studio 2 PRO.

```

1  #include <iostream> // 输入输出流
2  #include <string> // 字符串
3  #include <vector> // 向量
4  #include <algorithm> // 算法
5  #include <map> // 映射
6  #include <set> // 集合
7  #include <stack> // 栈
8  #include <queue> // 队列
9  #include <deque> // 双端队列
10 #include <list> // 链表
11 #include <array> // 数组
12 #include <tuple> // 元组
13 #include <unordered_map> // 无序映射
14 #include <unordered_set> // 无序集合
15 #include <memory> // 内存管理
16 #include <random> // 随机数
17 #include <chrono> // 时间
18 #include <thread> // 多线程
19 #include <atomic> // 原子操作
20 #include <mutex> // 互斥锁
21 #include <semaphore> // 信号量
22 #include <barrier> // 屏障
23 #include <condition_variable> // 条件变量
24 #include <future> // 未来
25 #include <promise> // 承诺
26 #include <weak_ptr> // 弱指针
27 #include <shared_ptr> // 共享指针
28 #include <weak_map> // 弱映射
29 #include <weak_set> // 弱集合
30 #include <weak_queue> // 弱队列
31 #include <weak_deque> // 弱双端队列
32 #include <weak_list> // 弱链表
33 #include <weak_array> // 弱数组
34 #include <weak_tuple> // 弱元组
35 #include <weak_unordered_map> // 弱无序映射
36 #include <weak_unordered_set> // 弱无序集合
37 #include <weak_memory> // 弱内存管理
38 #include <weak_random> // 弱随机数
39 #include <weak_chrono> // 弱时间
40 #include <weak_thread> // 弱多线程
41 #include <weak_atomic> // 弱原子操作
42 #include <weak_mutex> // 弱互斥锁
43 #include <weak_semaphore> // 弱信号量
44 #include <weak_barrier> // 弱屏障
45 #include <weak_condition_variable> // 弱条件变量
46 #include <weak_future> // 弱未来
47 #include <weak_promise> // 弱承诺
48 #include <weak_weak_ptr> // 弱弱指针
49 #include <weak_shared_ptr> // 弱共享指针
50 #include <weak_weak_map> // 弱弱映射
51 #include <weak_weak_set> // 弱弱集合
52 #include <weak_weak_queue> // 弱弱队列
53 #include <weak_weak_deque> // 弱弱双端队列
54 #include <weak_weak_list> // 弱弱链表
55 #include <weak_weak_array> // 弱弱数组
56 #include <weak_weak_tuple> // 弱弱元组
57 #include <weak_weak_unordered_map> // 弱弱无序映射
58 #include <weak_weak_unordered_set> // 弱弱无序集合
59 #include <weak_weak_memory> // 弱弱内存管理
60 #include <weak_weak_random> // 弱弱随机数
61 #include <weak_weak_chrono> // 弱弱时间
62 #include <weak_weak_thread> // 弱弱多线程
63 #include <weak_weak_atomic> // 弱弱原子操作
64 #include <weak_weak_mutex> // 弱弱互斥锁
65 #include <weak_weak_semaphore> // 弱弱信号量
66 #include <weak_weak_barrier> // 弱弱屏障
67 #include <weak_weak_condition_variable> // 弱弱条件变量
68 #include <weak_weak_future> // 弱弱未来
69 #include <weak_weak_promise> // 弱弱承诺
70 #include <weak_weak_weak_ptr> // 弱弱弱指针
71 #include <weak_weak_shared_ptr> // 弱弱共享指针
72 #include <weak_weak_weak_map> // 弱弱弱映射
73 #include <weak_weak_weak_set> // 弱弱弱集合
74 #include <weak_weak_weak_queue> // 弱弱弱队列
75 #include <weak_weak_weak_deque> // 弱弱弱双端队列
76 #include <weak_weak_weak_list> // 弱弱弱链表
77 #include <weak_weak_weak_array> // 弱弱弱数组
78 #include <weak_weak_weak_tuple> // 弱弱弱元组
79 #include <weak_weak_weak_unordered_map> // 弱弱弱无序映射
80 #include <weak_weak_weak_unordered_set> // 弱弱弱无序集合
81 #include <weak_weak_weak_memory> // 弱弱弱内存管理
82 #include <weak_weak_weak_random> // 弱弱弱随机数
83 #include <weak_weak_weak_chrono> // 弱弱弱时间
84 #include <weak_weak_weak_thread> // 弱弱弱多线程
85 #include <weak_weak_weak_atomic> // 弱弱弱原子操作
86 #include <weak_weak_weak_mutex> // 弱弱弱互斥锁
87 #include <weak_weak_weak_semaphore> // 弱弱弱信号量
88 #include <weak_weak_weak_barrier> // 弱弱弱屏障
89 #include <weak_weak_weak_condition_variable> // 弱弱弱条件变量
90 #include <weak_weak_weak_future> // 弱弱弱未来
91 #include <weak_weak_weak_promise> // 弱弱弱承诺
92 #include <weak_weak_weak_weak_ptr> // 弱弱弱弱指针
93 #include <weak_weak_weak_shared_ptr> // 弱弱弱共享指针
94 #include <weak_weak_weak_weak_map> // 弱弱弱弱映射
95 #include <weak_weak_weak_weak_set> // 弱弱弱弱集合
96 #include <weak_weak_weak_weak_queue> // 弱弱弱弱队列
97 #include <weak_weak_weak_weak_deque> // 弱弱弱弱双端队列
98 #include <weak_weak_weak_weak_list> // 弱弱弱弱链表
99 #include <weak_weak_weak_weak_array> // 弱弱弱弱数组
100 #include <weak_weak_weak_weak_tuple> // 弱弱弱弱元组
101 #include <weak_weak_weak_weak_unordered_map> // 弱弱弱弱无序映射
102 #include <weak_weak_weak_weak_unordered_set> // 弱弱弱弱无序集合
103 #include <weak_weak_weak_weak_memory> // 弱弱弱弱内存管理
104 #include <weak_weak_weak_weak_random> // 弱弱弱弱随机数
105 #include <weak_weak_weak_weak_chrono> // 弱弱弱弱时间
106 #include <weak_weak_weak_weak_thread> // 弱弱弱弱多线程
107 #include <weak_weak_weak_weak_atomic> // 弱弱弱弱原子操作
108 #include <weak_weak_weak_weak_mutex> // 弱弱弱弱互斥锁
109 #include <weak_weak_weak_weak_semaphore> // 弱弱弱弱信号量
110 #include <weak_weak_weak_weak_barrier> // 弱弱弱弱屏障
111 #include <weak_weak_weak_weak_condition_variable> // 弱弱弱弱条件变量
112 #include <weak_weak_weak_weak_future> // 弱弱弱弱未来
113 #include <weak_weak_weak_weak_promise> // 弱弱弱弱承诺
114 #include <weak_weak_weak_weak_weak_ptr> // 弱弱弱弱弱指针
115 #include <weak_weak_weak_weak_shared_ptr> // 弱弱弱弱共享指针
116 #include <weak_weak_weak_weak_weak_map> // 弱弱弱弱弱映射
117 #include <weak_weak_weak_weak_weak_set> // 弱弱弱弱弱集合
118 #include <weak_weak_weak_weak_weak_queue> // 弱弱弱弱弱队列
119 #include <weak_weak_weak_weak_weak_deque> // 弱弱弱弱弱双端队列
120 #include <weak_weak_weak_weak_weak_list> // 弱弱弱弱弱链表
121 #include <weak_weak_weak_weak_weak_array> // 弱弱弱弱弱数组
122 #include <weak_weak_weak_weak_weak_tuple> // 弱弱弱弱弱元组
123 #include <weak_weak_weak_weak_weak_unordered_map> // 弱弱弱弱弱无序映射
124 #include <weak_weak_weak_weak_weak_unordered_set> // 弱弱弱弱弱无序集合
125 #include <weak_weak_weak_weak_weak_memory> // 弱弱弱弱弱内存管理
126 #include <weak_weak_weak_weak_weak_random> // 弱弱弱弱弱随机数
127 #include <weak_weak_weak_weak_weak_chrono> // 弱弱弱弱弱时间
128 #include <weak_weak_weak_weak_weak_thread> // 弱弱弱弱弱多线程
129 #include <weak_weak_weak_weak_weak_atomic> // 弱弱弱弱弱原子操作
130 #include <weak_weak_weak_weak_weak_mutex> // 弱弱弱弱弱互斥锁
131 #include <weak_weak_weak_weak_weak_semaphore> // 弱弱弱弱弱信号量
132 #include <weak_weak_weak_weak_weak_barrier> // 弱弱弱弱弱屏障
133 #include <weak_weak_weak_weak_weak_condition_variable> // 弱弱弱弱弱条件变量
134 #include <weak_weak_weak_weak_weak_future> // 弱弱弱弱弱未来
135 #include <weak_weak_weak_weak_weak_promise> // 弱弱弱弱弱承诺
136 #include <weak_weak_weak_weak_weak_weak_ptr> // 弱弱弱弱弱弱指针
137 #include <weak_weak_weak_weak_weak_shared_ptr> // 弱弱弱弱弱共享指针
138 #include <weak_weak_weak_weak_weak_weak_map> // 弱弱弱弱弱弱映射
139 #include <weak_weak_weak_weak_weak_weak_set> // 弱弱弱弱弱弱集合
140 #include <weak_weak_weak_weak_weak_weak_queue> // 弱弱弱弱弱弱队列
141 #include <weak_weak_weak_weak_weak_weak_deque> // 弱弱弱弱弱弱双端队列
142 #include <weak_weak_weak_weak_weak_weak_list> // 弱弱弱弱弱弱链表
143 #include <weak_weak_weak_weak_weak_weak_array> // 弱弱弱弱弱弱数组
144 #include <weak_weak_weak_weak_weak_weak_tuple> // 弱弱弱弱弱弱元组
145 #include <weak_weak_weak_weak_weak_weak_unordered_map> // 弱弱弱弱弱弱无序映射
146 #include <weak_weak_weak_weak_weak_weak_unordered_set> // 弱弱弱弱弱弱无序集合
147 #include <weak_weak_weak_weak_weak_weak_memory> // 弱弱弱弱弱弱内存管理
148 #include <weak_weak_weak_weak_weak_weak_random> // 弱弱弱弱弱弱随机数
149 #include <weak_weak_weak_weak_weak_weak_chrono> // 弱弱弱弱弱弱时间
150 #include <weak_weak_weak_weak_weak_weak_thread> // 弱弱弱弱弱弱多线程
151 #include <weak_weak_weak_weak_weak_weak_atomic> // 弱弱弱弱弱弱原子操作
152 #include <weak_weak_weak_weak_weak_weak_mutex> // 弱弱弱弱弱弱互斥锁
153 #include <weak_weak_weak_weak_weak_weak_semaphore> // 弱弱弱弱弱弱信号量
154 #include <weak_weak_weak_weak_weak_weak_barrier> // 弱弱弱弱弱弱屏障
155 #include <weak_weak_weak_weak_weak_weak_condition_variable> // 弱弱弱弱弱弱条件变量
156 #include <weak_weak_weak_weak_weak_weak_future> // 弱弱弱弱弱弱未来
157 #include <weak_weak_weak_weak_weak_weak_promise> // 弱弱弱弱弱弱承诺
158 #include <weak_weak_weak_weak_weak_weak_weak_ptr> // 弱弱弱弱弱弱弱指针
159 #include <weak_weak_weak_weak_weak_weak_shared_ptr> // 弱弱弱弱弱弱共享指针
160 #include <weak_weak_weak_weak_weak_weak_weak_map> // 弱弱弱弱弱弱弱映射
161 #include <weak_weak_weak_weak_weak_weak_weak_set> // 弱弱弱弱弱弱弱集合
162 #include <weak_weak_weak_weak_weak_weak_weak_queue> // 弱弱弱弱弱弱弱队列
163 #include <weak_weak_weak_weak_weak_weak_weak_deque> // 弱弱弱弱弱弱弱双端队列
164 #include <weak_weak_weak_weak_weak_weak_weak_list> // 弱弱弱弱弱弱弱链表
165 #include <weak_weak_weak_weak_weak_weak_weak_array> // 弱弱弱弱弱弱弱数组
166 #include <weak_weak_weak_weak_weak_weak_weak_tuple> // 弱弱弱弱弱弱弱元组
167 #include <weak_weak_weak_weak_weak_weak_weak_unordered_map> // 弱弱弱弱弱弱弱无序映射
168 #include <weak_weak_weak_weak_weak_weak_weak_unordered_set> // 弱弱弱弱弱弱弱无序集合
169 #include <weak_weak_weak_weak_weak_weak_weak_memory> // 弱弱弱弱弱弱弱内存管理
170 #include <weak_weak_weak_weak_weak_weak_weak_random> // 弱弱弱弱弱弱弱随机数
171 #include <weak_weak_weak_weak_weak_weak_weak_chrono> // 弱弱弱弱弱弱弱时间
172 #include <weak_weak_weak_weak_weak_weak_weak_thread> // 弱弱弱弱弱弱弱多线程
173 #include <weak_weak_weak_weak_weak_weak_weak_atomic> // 弱弱弱弱弱弱弱原子操作
174 #include <weak_weak_weak_weak_weak_weak_weak_mutex> // 弱弱弱弱弱弱弱互斥锁
175 #include <weak_weak_weak_weak_weak_weak_weak_semaphore> // 弱弱弱弱弱弱弱信号量
176 #include <weak_weak_weak_weak_weak_weak_weak_barrier> // 弱弱弱弱弱弱弱屏障
177 #include <weak_weak_weak_weak_weak_weak_weak_condition_variable> // 弱弱弱弱弱弱弱条件变量
178 #include <weak_weak_weak_weak_weak_weak_weak_future> // 弱弱弱弱弱弱弱未来
179 #include <weak_weak_weak_weak_weak_weak_weak_promise> // 弱弱弱弱弱弱弱承诺
180 #include <weak_weak_weak_weak_weak_weak_weak_weak_ptr> // 弱弱弱弱弱弱弱弱指针
181 #include <weak_weak_weak_weak_weak_weak_weak_shared_ptr> // 弱弱弱弱弱弱弱共享指针
182 #include <weak_weak_weak_weak_weak_weak_weak_weak_map> // 弱弱弱弱弱弱弱弱映射
183 #include <weak_weak_weak_weak_weak_weak_weak_weak_set
```